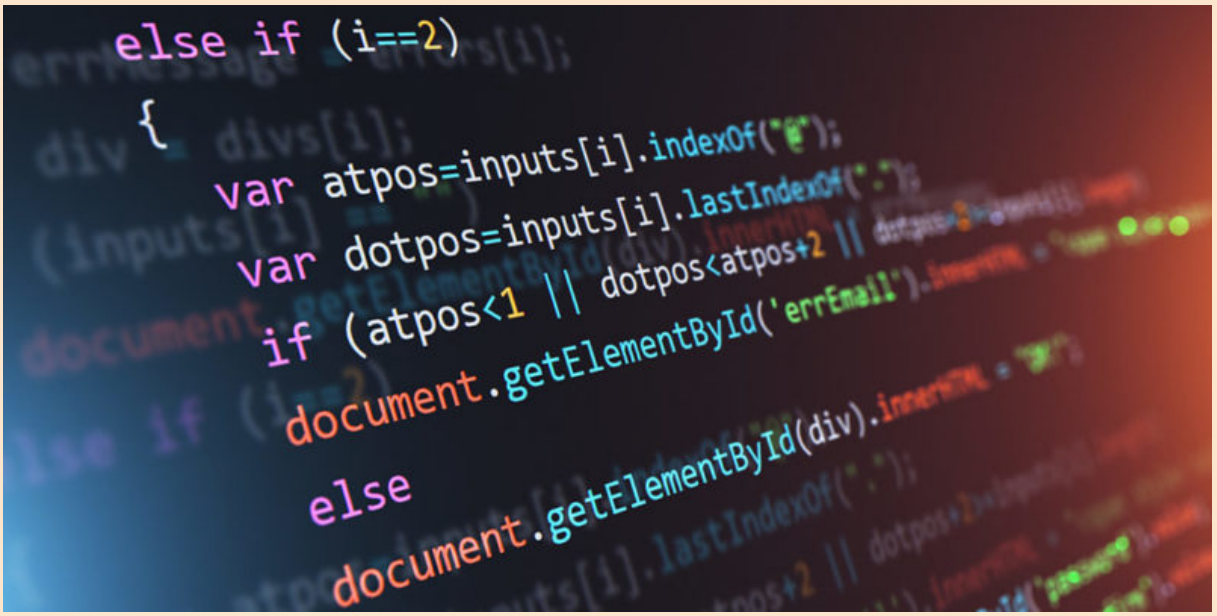
Les físiques en els videojocs



Oriol Vinuesa Sueiras

Institut Arquitecte Manuel Raspall

Àmbit: Física i Informàtica

Tutor: Oriol González - Agustín Sánchez

2n Batxillerat A

Curs: 2022 - 2023

Resumen

La física y la programación son dos ámbitos bastante relacionados en muchos aspectos. Mi principal motivación para ejecutar este trabajo es la pasión por la física y la informática. Este trabajo de investigación tiene como objetivo encontrar los principios físicos (cinemática principalmente) utilizados en la programación de simuladores o videojuegos y, además, verificar que se aplican correctamente.

El trabajo contiene todas las explicaciones que comprueban la correcta aplicación de la física en la programación de ciertos videojuegos y, también contiene, como producto final, cinco simuladores de físicas programados desde cero.

Abstract

Physics and programming are two fields that are quite related in many aspects. My principal motivation for this research work is my passion for physics and programming. The objective of this research work is to find those physical principles (mainly kinematics) used in the programming of simulators or video games and, in addition, verifying that they are applied correctly.

The research work contains all the explanations that prove the good application of physics in some video games and, also contains, as a final product, five simulators programmed from scratch.

Índex

1. Introducció……9

1.1. Objectius del treball……..9

1.2. Motivacions……..……...9

1.3. Metodologia……9

2. Evolució històrica dels videojocs……...10

3. Aprenentatge de llenguatges de programació……13

3.1. HTML……..……...……13

3.2. CSS……..……...14

3.3. JavaScript……..……...……15

3.3.1. Descripció general de JavaScript……..……15

3.3.2. Instruccions utilitzades en JavaScript……..……..16

3.3.2.1. function()....16

3.3.2.2. for()....17

3.3.2.3. if() - else()....17

3.3.2.4.. setInterval() - clearInterval()....18

4. Física i programació……..……..19

4.1. MRU/MCU.……..19

4.1.1. Explicació del principi físic……19

4.1.1.1. MRU……..……..19

4.1.1.2. MCU……..……..21

4.1.2 Programació del principi físic……..……..22

4.1.2.1. MRU……..……..22

4.1.2.2. MCU……..23

4.1.3. Aplicació del principi físic en videojocs……..……..24

4.1.3.1. Tap on Time……..…….……24

4.1.3.2. Pac-Man……...26

4.1.3.3. Geometry Dash……..30

4.2. MRUA/MCUA + Tir parabòlic en 2D i 3D.....30

4.2.1. Explicació del principi físic (MRUA/MCUA).....30

4.2.2. Programació del principi físic (MRUA/MCUA).....34

4.2.3. Explicació del principi físic (Tir parabòlic en 2D i 3D).....35

4.2.4. Programació del principi físic (Tir parabòlic en 2D i 3D).....37

4.2.5. Aplicació dels principis físics en videojocs……..38

4.2.5.1. Angry Birds 2 (tir parabòlic).....38

4.2.5.2. Cannon Balls 3D (tir parabòlic).....42

4.2.5.3. Jetpack Joyride (MRUA).....43

4.3. Conservació de P (Xocs).....44

4.3.1. Explicació del principi físic……..44

4.3.2. Programació del principi físic……..46

4.3.3. Aplicació del principi físic en videojocs……..47

4.3.3.1. Pool Billiards Pro……47

4.3.3.2. Angry Birds 2 i Cannon Balls 3D……48

5. Conclusions……49

6. Webgrafia……..50

Índex de figures

Figura 1 [Pàg 10]. Primer videojoc de la història creat per Alexander S.Douglas (1955).

Figura 2 [Pàg 10]. Foto d'una sala arcade, popularitzades entre 1971 i 1975.

Figura 3 [Pàg 11]. Portada del videojoc "Doom", un dels primers que van aplicar la tercera dimensió. Joc dissenyat per "id software" (1993).

Figura 4 [Pàg 12]. Evolució gràfica dels videojocs al llarg del temps.

FIgura 5 [Pàg 22]. Captures del simulador de MRU.

Figura 6 [Pàg 23]. Captura del simulador de MCU/MCUA.

Figura 7 [Pàg 27]. Passadís inferior del laberint de Pac-Man, utilitzat per diversos càlculs.

FIgura 8 [Pàg 28]. Passadís central del laberint de Pac-Man, utilitzat per diversos càlculs.

Figura 9 [Pàg 34]. Captures del simulador de MRUA.

Figura 10 [Pàg 35]. Captura del simulador de MCU/MCUA.

FIgura 11 [Pàg 36]. Sistema de referència x-y per a tirs parabòlics 2D.

Figura 12 [Pàg 26]. Descomposició d'un vector en les seves components x-y.

Figura 13 [Pàg 38]. Captura del simulador de tir parabòlic.

Figura 14 [Pàg 40]. Gràfiques del moviment x i del moviment y per el llançament parabòlic de l'ocell vermell d'Angry Birds 2. (extretes de l'aplicació "Tracker").

Figura 15 [Pàg 40]. Taula amb totes les dades de les gràfiques de la “Fig. 14.”. (extreta de “Tracker”):

Figura 16 [Pàg 40]. Captura de “Cannon Balls 3D”.

Figura 17 [Pàg 47]. Captura del simulador de xocs en 1D.

Figura 18 [Pàg 48]. Captura de “Pool Billiards Pro”:

1. Introducció

1.1. Objectius del treball

Els objectius principals del meu treball són els següents:

1. Analitzar si els videojocs contenen principis físics.
2. Aprendre com es programen en codi aquests principis físics.
3. Comprovar si els videojocs que apliquen principis físics els apliquen correctament.
4. Programar simuladors dels principis físics estudiats.

1.2. Motivacions

La principal motivació que m'ha portat a investigar sobre la física aplicada en videojocs és la meva passió per la física i la informàtica. A més a més, per tal de programar correctament, també s'han d'aplicar lleugerament idees matemàtiques, les quals formen part d'una matèria interessant i motivadora per a mi.

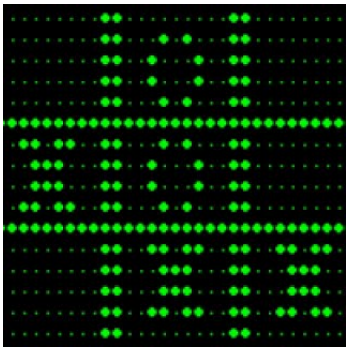
1.3. Metodologia

Primerament, he resumit a grans trets l'evolució dels videojocs al llarg de la història. Per la resta del meu treball seguiré una estructura i un ordre específic per parlar de cada principi físic. Per començar explicaré de què tracta el principi físic i d'on surten les equacions que el descriuen, després ho relacionaré amb la informàtica, explicaré com es fa per programar el principi físic i adjuntaré un simulador programat per mi. Per últim, buscaré exemples del principi físic en videojocs i comprovaré si està ben aplicat.

2. Evolució històrica dels videojocs (a grans trets)

Els inicis. (1952 - 1972)

Els primers videojocs van sorgir fa unes sis dècades, cap a l'any 1955. Els jocs més antics que es coneixen són jocs com el "Pong" o "OXO (tres en ratlla)". El "Pong", per exemple, es tracta d'un joc on una pilota viatja d'un extrem a l'altre de la pantalla i va rebotant amb unes barres que es poden moure amunt i avall. La trajectòria descrita per la pilota quan creua la pantalla es pot interpretar com un "Moviment Rectilini Uniforme".



Pel que fa a "OXO", es considera el primer joc de la història i va ser desenvolupat per Alexander S.Douglas. Tenia un aspecte del tot peculiar.

Fig. 1. "OXO" per Alexander S.Douglas.

La dècada dels 8 bits. (1973 - 1989)

Durant aquesta dècada es van obrir les primeres sales recreatives, en les quals, es podien trobar multitud de videojocs que ara es recorden com els "clàssics" o "jocs arcade*". Aquests jocs continuaven sent bastant simples, eren en dues dimensions.



Com a exemples podria posar "Pac-Man", en el qual una bola groga es mou sempre a la mateixa velocitat per un pla en dues dimensions, "Donkey Kong", que llença barrils que rodolen o "Asteroids", en el qual diferents meteorits viatgen per la pantalla.

Fig. 2. Foto d'una sala recreativa.

Cap a l'any 1983, l'empresa "Nintendo" va revolucionar el mercat amb la seva primera consola "NES" la qual podia executar jocs com "Super Mario Bros", un joc que va ser una gran novetat perquè l'objectiu ja no era passar una pantalla i una altra, totes iguals i intentar fer el màxim de punts sinó que ara havies de viatjar per diferents escenaris per aconseguir completar el videojoc. La mateixa idea va ser adoptada per moltes altres empreses com "SEGA".

* Un "joc arcade" és un tipus de videojoc que consta d'un conjunt de controls bàsics (esquerra, dreta, amunt avall, salta, agafa…) i està destinat a jugar partides curtes, entretingudes i repetitives.

Al cap de poc ja hi havia al mercat tota mena de videojocs. Tot i això, de 1983 a 1987 l'empresa que més va destacar va ser "Nintendo". L'any 1987 "Nintendo" va llançar al mercat una consola molt innovadora i portàtil: la Game Boy.

La revolució de les 3 dimensions. (1990 - 2000)

Cap als anys 90, les videoconsoles van fer un salt tècnic impressionant, ja que van deixar d'utilitzar processadors de 8 bits i van començar a utilitzar processadors de 16 bits, els quals donaven multitud de possibilitats noves. Aquestes millores van augmentar l'interès del públic i molta més gent va començar a jugar videojocs.

Mentre les empreses especialitzades en consoles gaudien d'aquesta millora tècnica que acabaven de fer i subministraven jocs cada vegada més interessants, altres empreses estaven treballant en videojocs d'entorn 3D principalment per PC. Un dels videojocs més destacables que va treballar la profunditat va ser "Doom 1993" juntament amb "Wolfenstein", videojocs en tres dimensions en els quals viatges per escenaris 3D mentre t'enfrontes a tota mena d'enemics.



Fig. 3. Portada del videojoc "Doom".

Les millores 3D no van trigar a arribar també a les consoles i "Nintendo", va llençar "SNES" una consola amb la qual podies jugar alguns jocs en entorns de 3 dimensions.

Després d'això "SEGA" que havia estat sempre perseguint a "Nintendo" va fer un gran salt, passant dels 16 bits als 32 bits amb consoles com la "Sega Saturn", la qual va oferir prestacions mai vistes i els videojocs van millorar considerablement. "Sony" va fer el mateix amb la "Sony PlayStation".

No gaire més tard van aparèixer consoles de 64 bits i els "jocs arcade" van començar a decaure mentre ocupaven el mercat els nous jocs en 3 dimensions. Una de les primeres consoles de 64 bits va ser la "Nintendo 64" que va oferir molts de jocs en 3 dimensions com "Mario 64" o "Zelda Ocarina of time".

Començament del nou segle. (2000 - Actualitat)

A principis de l'any 2000, "Sony" va llençar la videoconsola "Sony PlayStation 2" i "Microsoft" també es va unir al món de les consoles amb la "Xbox" l'any 2001.

Paral·lelament, “Nintendo” va llençar la successora de la “Nintendo 64”: la “Gamecube” i “SEGA”, com que no podia competir en aquest mercat, va deixar de produir hardware (maquinari) i es va especialitzar en software (programari) l’any 2002. Pel que fa als ordinadors, eren més flexibles que les consoles pel fet que se’ls podia canviar els components per millorar-los constantment i això va ajudar al ràpid avenç dels videojocs fins avui en dia. A més es van començar a produir perifèrics com volants i comandaments que es podien connectar al PC i oferien experiències semblants a les que oferien les consoles.



Fig. 4. Evolució gràfica dels videojocs al llarg del temps.

3. Aprenentatge de programació necessària pel treball

Per tal de poder veure clarament el fonament informàtic de la programació de físiques i poder-les programar vaig decidir aprendre els llenguatges de programació HTML, CSS i JavaScript. Amb aquests llenguatges puc escriure els codis de programació, però també necessito un entorn gràfic que els pugui executar (Un "entorn gràfic" és un conjunt de programes que transformen el codi, en forma de text, en una representació gràfica amb la qual ens entenem millor i amb la qual podem interactuar més fàcilment) i com que aquests tres llenguatges són acceptats per navegadors com "Google Chrome" o "FireFox", vaig decidir que utilitzaria un navegador com a entorn gràfic.

Per tal de poder executar amb un navegador els meus programes el que faig és escriure el codi en un document .txt (que pot ser per exemple un document escrit en l'aplicació "Bloc de Notes") i després canvio l'extensió a .html per tal que el que he escrit s'executi amb el navegador.

3.1. HTML

El primer llenguatge que vaig aprendre va ser HTML (HyperText Markup Language). Aquest llenguatge va ser inventat pel físic Tim Berners-Lee l'any 1980 aproximadament i és un llenguatge basat en etiquetes d'hipertext. Tot i que ja estigués inventat, HTML no va ser un llenguatge universal oficial fins que la IETF (Internet Engineering Task Force) el va declarar oficial l'any 1993.

HTML em serveix per controlar l'estructura del programa. Amb ell puc crear objectes, posar títols, posar "inputs" (que són entrades d'informació), posar imatges i un munt de coses més. HTML es basa en l'utilització d'etiquetes. Sempre que vulguem donar una informació al navegador posarem <...> i després tancarem amb </…> per exemple, si volem posar un títol hem d'utilitzar la següent etiqueta: <title>...</title> i a dins (on els tres puntets) el títol que vulguem que es vegi.

Un programa d'HTML sempre s'ha d'escriure dins l'etiqueta: <html>...</html> per tal que l'entorn gràfic que l'executi sàpiga de quin llenguatge es tracta. Després, dins d'aquesta etiqueta hi han d'haver també l'etiqueta <head>...</head> i la <bopy>...</body>.
La <head>...</head> s'utilitza per declarar variables i tot d'informacions (estil dels objectes o comportament dels mateixos) que es desen en la memòria quan el navegador llegeix el programa i la <bopy>...</body> és en la qual es creen els objectes, títols i "inputs" (els quals es podran visualitzar en executar el programa).

3.2. CSS

Paral·lelament a HTML vaig aprendre el llenguatge de programació CSS (Cascading Style Sheets). Aquest llenguatge va ser desenvolupat per W3C (World Wide Web Consortium), una organització centrada en el desenvolupament de documents electrònics. Quan HTML va començar a guanyar popularitat W3C va trobar necessari crear un llenguatge complementari a HTML per tal de poder canviar amb més facilitat l'estil dels documents electrònics i l'any 1995-1996 va crear CHSS (Cascading HTML Style Sheets) i posteriorment el seu nom va ser modificat a CSS (Cascading Style Sheets).

CSS em serveix per controlar l'estètica dels objectes, títols, inputs i tot el que pugui posar amb HTML. Quan dic estètica em refereixo a la forma, el color, l'opacitat, la posició… La part en CSS del codi HTML l'escriuré entre les etiquetes <head><style type="text/CSS">...</style></head>.
Escric *type="text/CSS"* perquè el navegador sàpiga que el que hi ha dins d'aquella etiqueta està escrit amb CSS i no amb HTML.
Per tal d'atribuir qualitats als elements que hi hagi en el <body>...</body> he de fer servir identificadors, si creo un objecte, per exemple, li he d'assignar un nom per després poder fer referència a ell en l'apartat de CSS:

```
<html>
    <head>
        <style type="text/CSS">
            #Objecte1{
                width: …; // Amplada de l'objecte.
                height: …; // Alçada de l'objecte.
                position: …; // Tipus de posició de l'objecte.
                background: …; // Fons de l'objecte.
```

```
                        border: …; // Voltant de l'objecte.
                    }
                </style>
            </head>
            <body>
                <div id="Objecte1"></div> // l'etiqueta <div> s'utilitza en HTML per crear objectes.
            </body>
</html>
```

Com es veu en aquest exemple, quan creem un element en el "body" li podem assignar un identificador si posem: id="..." i fer referència a ell en l'apartat de CSS si posem: #...

Quan el navegador llegeix l'exemple de la pàgina anterior, es troba primer amb les qualitats de l'objecte, abans que l'objecte estigui creat, per tant, el navegador guarda la informació en la memòria i quan arriba al "body" i ha de crear un objecte amb el mateix identificador que el que ha llegit abans, li assigna tot el que té guardat en la memòria.

És com si ens donessin un full en blanc i ens diguessin: si en girar el full us trobeu amb un quadrat, el pinteu de blau, i si us trobeu amb una rodona, la pinteu de vermell. Llavors girem el full i segons quina figura veiem la pintem del color que li correspongui.

A part d'assignar qualitats a objectes en particular també podem assignar qualitats a grups d'objectes. Per a fer-ho en comptes d'utilitzar id="..." utilitzem class="..." i en l'apartat de CSS no posem #... sinó **.**…

Si en el body tenim 4 objectes amb class="Grup1" tots ells tindran les qualitats que escrivim dins de **.**Grup1{...}.

3.3. JavaScript

3.3.1. Descripció general

JavaScript és el llenguatge més complicat dels tres, va ser ideat l'any 1995 per un desenvolupador de Netscape (un navegador web utilitzat entre 1994 i 2003) que es deia Brendan Eich. Aquest llenguatge serveix per programar el comportament dels elements que hi ha en el "body" d'una plana web. La part en JavaScript del codi HTML s'escriu de manera semblant a la part en CSS, ja que també es posa en el "head" de la manera que es pot veure a la següent pàgina.

<head><script type=”text/javascript”>...</script></head>.
Igual que per CSS, declaro el type com a text/javascript per indicar quin llenguatge de programació estic fent servir.

3.3.2. Instruccions utilitzades en JavaScript

3.3.2.1. function()

JavaScript és un llenguatge bastant lliure. Per programar qualsevol acció, com per exemple que un cub avanci, puc fer servir la instrucció següent: function avança(){...} i entre els claudàtors escric el que vull que passi quan aquesta funció s’activi (vull que el cub avanci). El nom de la funció és arbitrari, també podria posar: function cub(){...}.

Com que les funcions en JavaScript les escrivim en el “head”, el navegador fa el mateix que amb les informacions en CSS: les guarda en memòria fins que li diem que les utilitzi. Una manera d’activar una funció és posar un botó en el “body” que activi la funció. Exemple a continuació:

```
<html>
    <head>
        <style type="text/css">
            #activador{
                position: absolute; // Indico que la posició és absoluta que vol dir que
                                    hauré d’especificar la seva posició respecte els
                                    marges de la pantalla.
                left: 50px; // Posició inicial respecte el marge esquerre de la pantalla.
                top: 50px; // Posició inicial respecte el marge superior de la pantalla.
            }
        </style>
        <script type="text/javascript">
            x=50; // declaro una variable de nom qualsevol que després utilitzaré per canviar la
                    posició del botó.
            function Començar(){ // funció de nom “Començar”
                x=x+10; // Quan la funció “Començar” s’activi el primer que farà és sumar
                          10 al valor de la variable “x”.
                document.getElementById("activador").style.left=x+"px";
                // Aquesta instrucció serveix per fer referència a un element, concretament a
                  la seva qualitat “left” de l’”style” i li estem dient que la posició left de
                  l’activador és igual al valor x +”px” ja que són les unitats que ha de tenir un
                  valor de left.
            }
        </script>
    </head>
```

```
    <body>
        <input id="activador" type="button" onclick="Començar()"></input>
        // Amb aquesta instrucció estic creant un “input” amb el nom “activador”, del tipus botó i que al
           clicar-lo activi la funció “Començar”.
    </body>
</html>
```

Resumidament, el programa anterior crea un botó que en clicar-lo suma 10 a una variable i després assigna a la posició del botó aquella variable, fent així que cada vegada que el cliquem es mogui 10 píxels a la dreta.

3.3.2.2. for()

A part de programar accions/funcions també podem programar bucles utilitzant una instrucció fonamental: for(){...}. Quan vulguem emprar aquesta funció hem d'escriure el següent, els números poden variar, però l'estructura ha de ser aquesta: for(i=0;i<10;i++){...} i posar entre els claudàtors el que vulguem que es repeteixi. En l'exemple que he escrit jo, el bulce es repetirà 10 vegades, ja que la variable "i" comença al valor 0 (i=0) i mentre "i" sigui més petita que 10 (i<10) el que hi hagi entre els claudàtors es repetirà, a més cada vegada que es completi el que hi hagi entre els claudàtors se li sumarà 1 a "i" (i++).

En efectes pràctics, ens podem imaginar que anem a una llibreria i només portem a la cartera monedes d'un euro. A l'hora de pagar un llibre de 10 euros podem agafar les monedes d'una en una i cada vegada que n'agafem una, comprovar si ja en tenim deu, i si és així, deixar d'agafar-ne; també podem agafar-ne de dues en dues, així només cal repetir l'acció d'agafar monedes 5 vegades. Sigui com sigui, es tracta de repetir la mateixa acció un nombre determinat de vegades fins a haver-ho repetit les vegades que ens interessen.

3.3.2.3. if() - else

Una altra instrucció important és: if(){…} la qual només executa el que hi ha entre els claudàtors si el que hi ha entre parèntesis es compleix. Per exemple: if(x==0){...}. En aquest cas, només es farà el que hi hagi entre claudàtors si x és igual a 0.
Un altre exemple: if(x>10){...}. En aquest, es farà el que hi hagi entre claudàtors si x és més gran que 10.

Relacionat amb la instrucció if(){...} existeix la instrucció else{...}, la qual es posa a continuació de la instrucció if(){...} i serveix per dir al programa què ha de fer si la condició entre parèntesis de if(){...}no es compleix.

If() es pot, com en el cas del for(), relacionar amb una situació quotidiana, o si més no, conceptual. Imaginem que passegem per un camí amb pomers a banda i banda, mentre passegem anem collint pomes i posant-les en un cistell. Quan hem caminat una bona estona ja tenim un nombre considerable de pomes, però llavors arribem a un encreuament on un senyor ens diu: “si teniu més de 20 pomes aneu pel camí de la vostra dreta, si no, preneu el de l’esquerra”. Llavors nosaltres fem un recompte de les pomes i seguim caminant pel camí que toqui.

3.3.2.4. setInterval() - clearInterval()

Per últim, dins de les instruccions que més utilitzaré, tenim la instrucció setInterval() la qual serveix per fer que una altra funció/instrucció es repeteixi cada cert temps, ens serà especialment útil, ja que ens serveix per simular el pas del temps, i això és fonamental en la programació de físiques. Per fer servir setInterval(), si jo tinc la funció: function cub(), puc posar: setInterval(cub,1000) i la funció cub() es repetirà cada segon. Quan fem ús de la instrucció setInterval() hem de posar entre els parèntesis quina funció es repeteix i cada quants milisegons ho fa.
Per tal d’anul·lar un setInterval() emprem clearInterval(). L’únic que cal fer és assignar a setInterval() un nom:

```
Temps=setInterval(cub,1000); // L’interval que repeteix la funció “cub” cada 1000 ms rep el nom de “Temps”.
```

I quan vulguem anul·lar-lo escrivim en el codi:

```
clearInterval(Temps); // Aquesta instrucció eliminarà l’interval que li indiquem, en aquest cas l’interval de nom “Temps” que hem creat anteriorment.
```

Les instruccions function(), for(), if(), else, setInterval() i clearInterval() són les que més utilitzaré al llarg de la programació de físiques.

4. Física i programació

4.1. MRU/MCU

4.1.1. Explicació del principi físic

4.1.1.1. MRU

Les lletres MRU corresponen a les inicials del fenomen físic anomenat “Moviment Rectilini Uniforme”. El MRU juntament amb el MRUA que explicaré més endavant, són dos principis físics fonamentals en la programació, ja que serveixen per descriure moviments simples. Per tal de programar-los necessitem unes equacions específiques anomenades funcions posició-temps les quals deduiré més endavant.

Diem que un cos segueix un MRU quan es mou a velocitat constant dibuixant una trajectòria recta. La velocitat és una magnitud que mesura el desplaçament per unitat de temps, per exemple, si un cos es mou a 7 m/s, vol dir que quan passa 1 segon el cos ha avançat 7 metres, per tant, en 3 segons haurà avançat 21 metres i en 4 segons 28 metres, etc.

La fórmula general de la velocitat és:

$$v = \frac{\Delta x}{\Delta t} \rightarrow v = \frac{x_f - x_o}{t_f - t_o}\ [m/s]$$

** On x_f és la posició final, x_0 és la posició inicial, t_o és el temps inicial i t_f és el temps final. (posició en metres i temps en segons).*

L’expressió anterior ens serveix en molts casos pràctics per determinar la velocitat mitjana entre dos instants de temps. Tot i això, existeix una definició més encertada de velocitat en la qual agafem un interval de temps molt petit, el màxim de petit que puguem:

$$v = \lim_{\Delta t \to 0} \frac{\Delta x}{\Delta t}$$

Per veure-ho més clar: si fem un trajecte de 120 km en 2 hores haurem dut una velocitat mitjana de 60 km/h. El que passa és que potser la primera hora de camí era de pujada i hem anat a 30 km/h i la segona hora hem anat a 90 km/h, fent així que la

velocitat mitjana del viatge segueixi sent 60 km/h, ja que també haurem fet 120 km en 2 hores. Ens podríem trobar que la velocitat fos diferent també cada mitja hora, o cada quart d'hora, o cada minut i així fins que les particions de temps fossin infinitament petites i la velocitat canvies en cada una d'aquestes particions. Tornant a l'expressió anterior:

$$v = \lim_{\Delta t \to 0} \frac{\Delta x}{\Delta t} \to v = \frac{dx}{dt}$$

** On el dt és la partició més petita que puguem fer del temps.*

A partir d'aquesta expressió podem arribar a una equació que ens dona la posició del cos que estiguem estudiant, per cada instant de temps. Aquesta és la que necessitem per a poder programar el principi físic en un videojoc. Per començar aïllem dx i fem una integral definida entre t_o i t_f (temps inicial i temps final) pel dt i entre x_o i x_f (posició inicial i posició final) pel dx:

$$v \cdot dt = dx$$

$$\int_{t_o}^{t_f} v \cdot dt = \int_{x_o}^{x_f} dx$$

Aplicant linealitat:

$$v \cdot \int_{t_o}^{t_f} dt = \int_{x_o}^{x_f} dx$$

Eliminant les integrals i els diferencials:

$$v \cdot |t|_{t_o}^{t_f} = |x|_{x_o}^{x_f}$$

Substituint els límits per la variable:

$$v \cdot (t_f - t_o) = (x_f - x_o)$$

Suposant que t_o és 0 i reorganitzant els termes:

$$x_f = x_o + v \cdot t_f$$

L'expressió del final de la pàgina anterior és la que ens serveix per calcular la posició respecte del temps i ens ajuda en la programació de moviments rectilinis, ja que l'ordinador l'utilitza per calcular la posició del cos que vulguem cada cert temps. Normalment, s'escriu com una funció i de la següent manera:

$$x(t) = x_o + v \cdot t$$

4.1.1.2. MCU

Abans de passar a veure com es programa aquest principi en videojocs, falta per dir que el MCU (Moviment Circular Uniforme) és el mateix que el MRU, però al llarg d'una circumferència, en la qual, el cos té una velocitat lineal i una velocitat angular (aquesta indica l'angle que descriu el cos per unitat de temps):

$$\omega = \frac{\Delta\varphi}{\Delta t} \rightarrow \omega = \frac{\varphi_f - \varphi_o}{t_f - t_o}$$

** On ω és la velocitat angular en radiants per segon i φ és l'angle descrit en radiants.*

La funció que ens diu quin arc de circumferència recorre el cos segons el temps, és la mateixa que en el MRU (fent servir la velocitat lineal). En canvi, si volem saber quin angle descriu el cos segons el temps, tenim una funció diferent (simplement canvien els termes, la idea és la mateixa):

$$\varphi(t) = \varphi_o + \omega \cdot t$$

** On ω és la velocitat angular en radiants per segon i φ és l'angle descrit en radiants.*

Si sabem quin angle ha descrit el cos i volem saber l'arc de circumferència que ha dibuixat (Δx), podem fer-ho a partir d'aquesta fórmula:

$$\Delta x = \Delta\varphi \cdot r$$

** On r és el radi de la circumferència en metres.*

Per últim, si volem saber la velocitat lineal i tenim la velocitat angular, fem servir la següent fórmula:

$$v = \omega \cdot r$$

** On ω és la velocitat angular en radiants per segon*

4.1.2. Programació del principi físic

4.1.2.1. MRU

Per tal de programar el MRU (el primer dels dos principis físics que acabem de veure) s'hauria de crear un "bucle" que repeteixi cada segon una funció la qual calculi la posició d'un objecte igual que ho faria la funció del MRU: $x(t) = x_o + v \cdot t$

Bàsicament, el programa actualitza la posició de l'objecte i li assigna la que ha calculat a partir de la funció posició-temps cada cert temps. Pot ser cada segon, cada mil·lisegon o cada minut.

Pel que fa al codi, utilitzarem la funció setInterval() i farem que cada segon el programa calculi la nova posició del cos i sumi 1 a una variable que farà de temps (en segons). Aquí adjunto un simulador de MRU que he programat: *https://sites.google.com/view/ovsmru/inicio*

Codi del Simulador a l'Annex 2.

Fotos del simulador:



Fig. 5. Captures del simulador de MRU.

4.1.2.2. MCU

A part del MRU tenim el MCU i per a programar-lo hem d'aplicar una idea diferent que la que apliquem en MRU, ja que hem d'utilitzar l'equació de la posició respecte d'un angle de rotació. Per aquest programa, el que farem és utilitzar la següent equació: $\varphi(t) = \varphi_o + \omega \cdot t$. Amb ella el joc calcularà quin angle s'ha mogut l'objecte i a partir d'aquest angle, sabent el radi del moviment que està descrivint i fent servir les funcions sinus i cosinus podrà saber la posició que ha de tenir el cos i assignar-la a les seves qualitats "left" i "top" (que defineixen la posició de l'objecte respecte dels marges de la pantalla). Igual que en el programa de MRU, la funció que calcula la posició de l'objecte es repetirà cada segon.

També he programat un simulador de MCU, que va juntament amb el simulador de MCUA el qual explicaré més endavant:
https://sites.google.com/view/mcumcua/inicio
Nota: per tal de fer un MCU no emplenar el requadre que posa "acceleració angular".

Codi del Simulador a l'Annex 3.
Fotos del simulador:

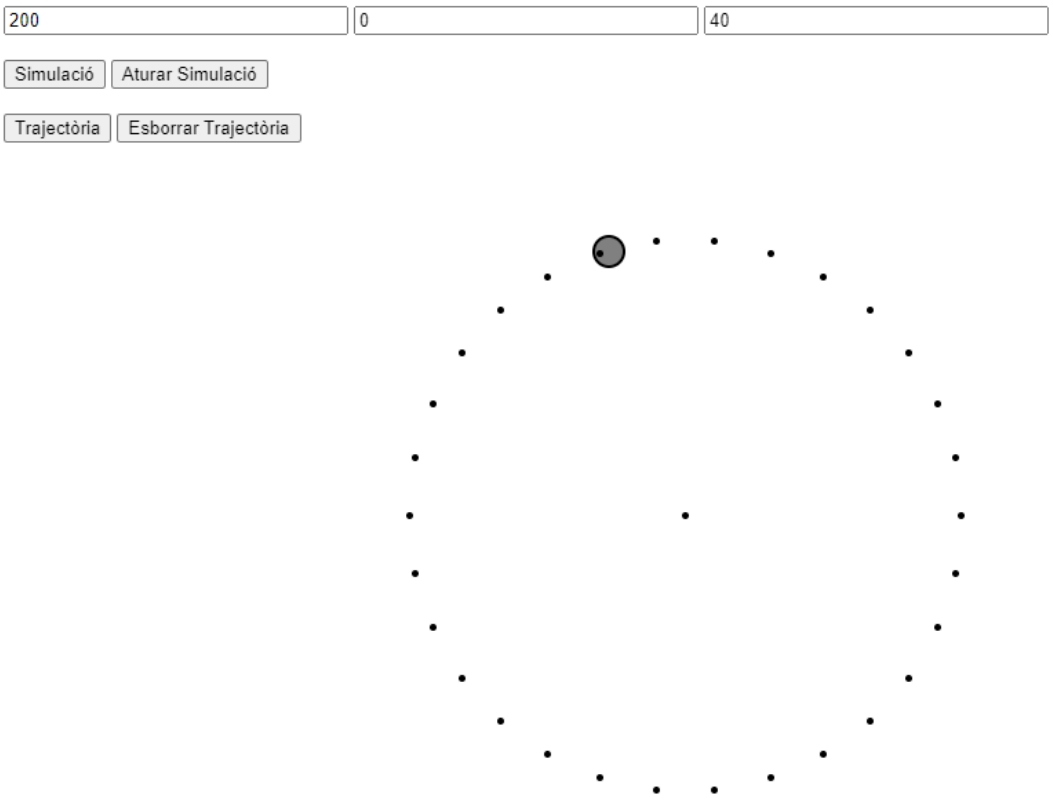


Fig. 6. Captura del simulador de MCU/MCUA

4.1.3. Aplicació del principi físic en videojocs

(Al llarg dels exemples prendré mesures amb cronòmetre i els assignaré un error de ± 0,1 segons, ja que la mesura no és 100% exacte.)

4.1.3.1. Tap on Time (MCU)

4.1.3.1.1. Descripció General del videojoc

Tap on Time és un videojoc per a dispositius mòbils en el qual una esfera es mou de manera circular, l'objectiu del joc és clicar la pantalla en el moment en què l'esfera passa per una zona fosca, la qual es troba en algun punt de la trajectòria circular que segueix l'esfera.

Per tal d'analitzar aquest videojoc he gravat un vídeo per veure com es juga al joc (primer QR) i un altre en el qual es pot apreciar fàcilment el principi físic (segon QR).

1: 2:

1:https://www.youtube.com/watch?v=z3tf1NHYwyk

2: https://www.youtube.com/watch?v=JYesDOq_iWE

4.1.3.1.2. Comprovació del principi físic que aplica

Per comprovar que el moviment de l'esfera és un MCU cronometraré quant triga a fer 5 voltes i quant triga a fer-ne 10:

Temps 5 voltes: 5,2 ± 0,1 segons. *(mesura presa amb el cronòmetre)*

Temps 10 voltes: 10,3 ± 0,1 segons. *(mesura presa amb el cronòmetre)*

Efectivament, es tracta d'un MCU, ja que $5,2 + 5,2 = 10,2$ $(10,3 - 0,1 = 10,2)$ i, per tant, el temps que l'esfera triga a fer 10 voltes és dues vegades el temps que triga a fer-ne 5. Per saber la velocitat que porta l'esfera:

$$\omega = \frac{\Delta\varphi\ (voltes)}{\Delta t} \rightarrow \omega_{10\ voltes} = \frac{10}{10,3} = 0,97\ voltes/segon$$

$$\omega_{5\ voltes} = \frac{5}{5{,}2} = 0{,}96\ voltes/segon$$

Aproximadament 1 volta per segon que en radiants és:

$$\frac{1\ volta}{1\ segon} \cdot \frac{2 \cdot \pi\ rad}{1\ volta} = 2\Pi\ rad/s$$

Per verificar que això és cert cronometraré quant triga a fer 8 voltes i després calcularé utilitzant les fórmules del MCU quan hauria de trigar i a veure si em coincideix:

Temps 8 voltes: 8,1 $\pm$ 0,1 segons. *(mesura presa amb el cronòmetre)*

"8 voltes són $16\boldsymbol{\pi}$ *radiants"*

Comprovació:

$$\varphi(t) = \varphi_o + \omega \cdot t$$

$$16\pi = 0 + 2\pi \cdot t \rightarrow t = \frac{16\pi}{2\pi} = 8\ segons$$

Utilitzant la velocitat angular i les fórmules del MCU podem saber l'angle que descriu l'esfera en qualsevol temps i el temps que triga a descriure qualsevol angle:

Voltes (angle descrit) que donaria en 60 segons:

$$\varphi(60) = 0 + 2\pi \cdot 60 = 120\pi\ rad \rightarrow 60\ voltes$$

Temps que trigaria en donar 17 voltes:

$$17\ voltes = 34\pi \rightarrow\ rad 34\pi = 0 + 2\pi \cdot t \rightarrow t = \frac{34\pi}{2\pi} = 17\ segons$$

També podríem buscar la velocitat lineal i els arcs de circumferència que descriu l'esfera, però per fer-ho s'ha de mesurar el radi del cercle pel qual es mou i depenent de la resolució de la pantalla o del dispositiu utilitzat, el radi varia i els càlculs serien diferents per cada dispositiu o cada resolució de pantalla.

En conclusió, he pogut provar que el moviment descrit per l'esfera és un MCU i que, per tant, el videojoc aplica bé la física darrere del "Moviment Circular Uniforme".

4.1.3.2. Pac-Man (MRU)

4.1.3.2.1. Descripció General del videojoc

Com a últim exemple de MRU/MCU he triat el Pac-Man, un videojoc antic que va ser molt important en el seu moment. El joc consisteix en menjar-se totes les boles taronges que hi ha repartides al llarg d'un laberint sense que t'atrapin 4 fantasmes que estan constantment fent voltes per aquest laberint.

El moviment que descriuen tant els fantasmes com el personatge principal, el Pac-Man, és un MRU. La gràcia és que podem canviar la direcció i el sentit del moviment per tal de poder-nos moure per tot el laberint.

La mecànica del Pac-Man és simple, va sempre recte a no ser que es trobi amb una paret, que llavors es queda aturat mirant cap a la paret i esperant que li ordenis cap a quina nova direcció ha d'anar, o a no ser que estigui anant per un camí recte i en passar pel costat d'un camí que gira (una bifurcació en la qual pot seguir recte o pot girar) cliques el botó de girar, llavors en Pac-Man canvia el seu rumb i se'n va pel nou camí.

La mecànica dels fantasmes és més complexa, ja que són quatre i cadascun funciona diferent. El més senzill és el fantasma vermell, el qual sempre segueix el rumb del Pac-Man. Els altres tenen patrons de seguiment més complexos i amb algunes aleatorietats programades expressament que fan que no puguis acabar de saber mai quin serà el seu pròxim moviment.

Existeix una última mecànica en el joc de Pac-Man, la qual es dona quan Pac-Man es menja una de les quatre boles grosses que hi ha repartides pel laberint. En fer-ho, la seva velocitat s'incrementa alhora que la dels fantasmes disminueix. A més els fantasmes canvien el seu color a un color blau i s'allunyen tant com poden de Pac-Man, ja que mentre l'efecte de la bola grossa duri, en Pac-Man es podrà menjar als fantasmes.

4.1.3.2.2. Anàlisi del moviment de “Pac-Man”

Per tal d’estudiar el MRU en aquest joc he gravat diversos vídeos i, per comprovar si el moviment que segueix Pac-Man és un MRU, aniré al passadís inferior del laberint i cronometraré quant triga Pac-Man en recórrer una meitat del passadís i quant triga en recórrer’l tot.



Fig. 7. Passadís inferior del laberint de Pac-Man, utilitzat per diversos càlculs.

També comptaré les boles taronges que es menja, ja que és el que faré servir com a unitat de distància.

(Jugabilitat) (Recopilació de moviments)

Jugabilitat: https://www.youtube.com/watch?v=ID79ieLu3Cw&t=13s

Recopilació de moviments: https://www.youtube.com/watch?v=J-z5fZJG9As

Pac-Man (sense l’efecte de la bola grossa)

Temps primer tram (12 boles): 1,7 $\pm$ 0,1 segons. *(mesura presa amb el cronòmetre)*

Temps passadís sencer (25 boles): 3,5 $\pm$ 0,1 segons. *(mesura presa amb el cronòmetre)*

Efectivament, es tracta d’un MRU, ja que $1,7 + 1,7 = 3,4$ $(3,5 - 0,1 = 3,4)$ i per tant el temps que Pac-Man triga a recórrer tot el passadís és dues vegades el temps que triga a recórrer el tram 1. Per saber la velocitat que porta:

$$v = \frac{\Delta x\ (boletes\ taronges\ menjades)}{\Delta t}$$

$$v_{tram\ 1} = \frac{12\ (la\ primera\ se\ la\ menja\ abans\ de\ començar)}{1{,}7} = 7,06\ boles/segon$$

$$v_{passadís\ sencer} = \frac{25}{3{,}5} = 7{,}14\ boles/segon$$

Per ser més precís faré la mitjana entre els dos valors obtinguts:

$$\frac{7{,}06+7{,}14}{2} = 7{,}1\ boles/segon$$

Ara, per verificar que el que acabo de fer és correcte, agafaré un altre passadís del laberint de Pac-Man, concretament aquest:

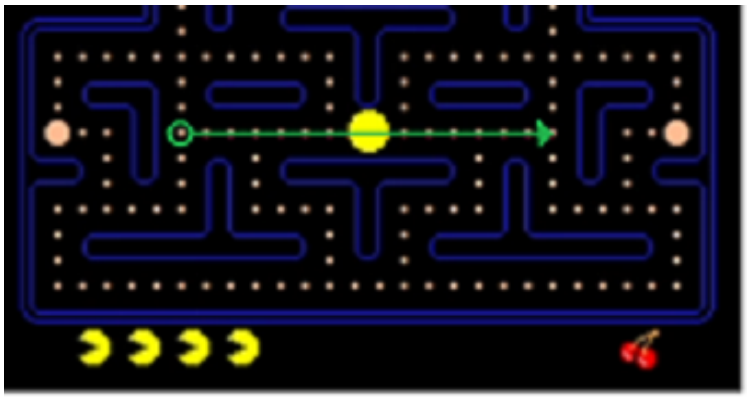


Fig. 8. Passadís central del laberint de Pac-Man, utilitzat per diversos càlculs.

Llavors cronometraré quant triga Pac-Man a recórre'l i a veure si coincideix amb el que hauria de trigar segons els càlculs (també comptaré les boles que es menja, ja que és el que estic utilitzant com a unitat de distància):
Temps nou passadís (14 boles): 2 $\pm$ 0,1 segons. *(mesura presa amb el cronòmetre)*

Comprovació:

$$x(t) = x_o + v \cdot t$$

$$14 = 0 + 7{,}1 \cdot t \rightarrow t = \frac{14}{7{,}1} = 1{,}97\ segons \simeq 2\ segons$$

La mesura presa amb el cronòmetre coincideix amb la calculada a partir de les equacions. Acabo de comprovar que la velocitat que he trobat anteriorment és correcta, i ara ja puc, amb les equacions del MRU, trobar quantes boles es menjaria Pac-Man per cada instant de temps o quant de temps trigaria a menjar-se qualsevol quantitat de boles.

Boles que es menjaria en 15 segons:

$$x(15) = 0 + 7{,}1 \cdot 15 = 106{,}5\ boles$$

$$\rightarrow 106\ boles\ (no\ es\ pot\ menjar\ mitja\ bola...)$$

Temps que trigaria en menjar-se 35 boles:

$$35 = 0 + 7,1 \cdot t \rightarrow t = \frac{35}{7,1} = 4,9\ segons$$

Pac-Man (sota l'efecte de la bola grossa)

Temps primer tram (12 boles): 1,6 $\pm$ 0,1 segons. *(mesura presa amb el cronòmetre)*

Temps passadís sencer (25 boles): 3,3 $\pm$ 0,1 segons. *(mesura presa amb el cronòmetre)*

Efectivament, també es tracta d'un MRU: $1,6 + 1,6 = 3,2$ $(3,3 - 0,1 = 3,2)$ i, per tant, el temps que Pac-Man triga a recórrer tot el passadís és dues vegades el temps que triga a recórrer el tram 1. Per saber la velocitat que porta:

$$v = \frac{\Delta x\ (boletes\ taronges\ menjades)}{\Delta t}$$

$$v_{tram\ 1} = \frac{12\ (la\ primera\ se\ la\ menja\ abans\ de\ començar)}{1,6} = 7,5\ boles/segon$$

$$v_{passadís\ sencer} = \frac{25}{3,3} = 7,6\ boles/segon$$

Per ser més precís faré la mitjana entre els dos valors obtinguts:

$$\frac{7,5+7,6}{2} = 7,55\ boles/segon$$

Ara, per verificar que el que acabo de fer és correcte, faré el mateix que en el cas del Pac-Man sense l'efecte de la bola grossa:

Temps nou passadís (14 boles): 1,8 $\pm$ 0,1 segons. *(mesura presa amb el cronòmetre)*

Comprovació:

$$x(t) = x_o + \nu \cdot t$$

$$14 = 0 + 7,55 \cdot t \rightarrow t = \frac{14}{7,55} = 1,85\ segons$$

La mesura presa amb el cronòmetre coincideix amb la calculada a partir de les equacions. Ara, igual que quan estava calculant pel Pac-Man sense l'efecte de la bola grossa, puc saber quant trigaria Pac-Man (sota l'efecte de la bola grossa) a menjar-se qualsevol quantitat de boles o quantes boles es podria menjar per qualsevol temps.

Boles que es menjaria en 15 segons:

$$x(15) = 0 + 7,55 \cdot 15 = 113,25\ boles$$

$$\rightarrow 113\ boles\ (no\ es\ pot\ menjar\ un\ quart\ de\ bola...)$$

Temps que trigaria en menjar-se 35 boles:

$$35 = 0 + 7,55 \cdot t \rightarrow t = \frac{35}{7,55} = 4,64\ segons$$

En conclusió, Pac-Man descriu dos tipus de MRU amb diferents velocitats cadascun depenent de si s'ha menjat o no una "bola grossa". En qualsevol dels dos casos, el principi físic està ben aplicat.

4.1.3.3. Geometry Dash (MRU)

A part de Tap on Time i Pac-Man, també he analitzat i observat un joc anomenat Geometry Dash, el qual aplica el principi del MRU. Després d'analitzar-lo he conclòs, igual que en el cas de Tap on Time i Pac-Man, que el principi estava ben aplicat. Tot el procediment i detalls es troben a l'annex 1.

4.2. MRUA/MCUA + Tir parabòlic en 2D i 3D

4.2.1. Explicació del principi físic (MRUA/MCUA)

Aquest fenomen físic és una modificació del MRU. Les inicials MRUA signifiquen "Moviment Rectilini Uniformement Accelerat" i MCUA "Moviment Circular Uniformement Accelerat".

La idea principal d'aquest principi físic és que la velocitat varia, pot augmentar o pot disminuir, però sempre ho fa de manera uniforme al llarg del temps. Mentre passa això, el cos descriu una trajectòria recta.

Per saber com augmenta o disminueix la velocitat existeix una magnitud anomenada acceleració que ens indica quants m/s varien per unitat de temps. La fórmula general de l'acceleració és:

$$a = \frac{\Delta v}{\Delta t} \rightarrow a = \frac{v_f - v_o}{t_f - t_o}\ \left[m/s^2\right]$$

** On v_f és la velocitat final i v_o la velocitat inicial.*

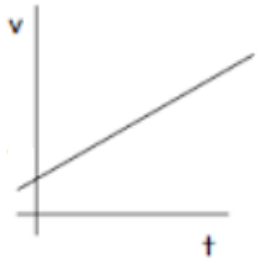
Per obtenir una fórmula que ens doni la posició en funció del temps no ho tenim tan fàcil com en el cas del MRU. Per començar, podem obtenir una fórmula que ens doni la velocitat en funció del temps a partir de la fórmula de l'acceleració:

$$a = \frac{v_f - v_o}{t_f - t_o} \rightarrow (t_f - t_o) \cdot a = v_f - v_o \rightarrow v_f = v_o + a \cdot (t_f - t_o)$$

Suposant que el temps inicial és 0:

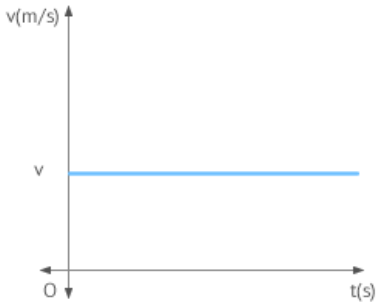
$$v(t) = v_o + a \cdot t$$

Si representem la funció $v(t)$ en una gràfica (suposant que a>0) obtenim el següent:



Com podem observar, la velocitat augmenta de manera uniforme a mesura que passa el temps.

En el MRU la gràfica que ens dona la velocitat en funció del temps és la següent:

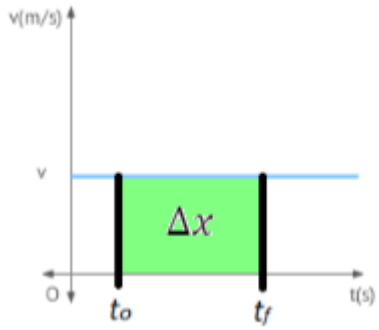


La velocitat no varia al llarg del temps.

Sabem que la velocitat multiplicada pel temps ens dona l'increment de posició, ja que:

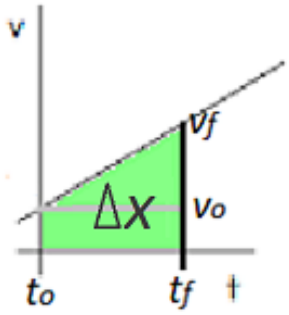
$$v = \frac{\Delta x}{\Delta t} \rightarrow \Delta x = v \cdot \Delta t$$

Per tant, en el gràfic, l'àrea compresa entre la funció i l'eix x és la posició:



$$(t_f - t_o) \cdot v = \Delta x$$

Podem generalitzar aquesta idea i aplicar-la en el MRUA també. En aquest cas l'àrea entre la funció i l'eix x està formada per un quadrilàter i un triangle (es podrien donar casos que estigués formada per dos triangles o fins i tot per un sol triangle, però la idea seria la mateixa: calcular l'àrea):



$$\Delta x = A\,triangle + A\,quadrilàter = \frac{(v_f - v_o)\cdot(t_f - t_o)}{2} + (t_f - t_o) \cdot v_o$$

Ara, combinant aquesta equació que hem obtingut del gràfic i la fórmula general de l'acceleració obtenim una funció que ens diu la posició per cada instant de temps tenint en compte l'acceleració (i sense haver de conèixer la velocitat final):

Primera (gràfic): $\Delta x = \frac{1}{2}(v_f - v_o) \cdot (t_f - t_o) + (t_f - t_o) \cdot v_o$

Segona (acceleració): $a = \frac{v_f - v_o}{t_f - t_o} \rightarrow v_f - v_o = a \cdot (t_f - t_o)$

Substituïm en la primera equació la segona:

$$\Delta x = \frac{1}{2} \cdot a \cdot (t_f - t_o) \cdot (t_f - t_o) + (t_f - t_o) \cdot v_o$$

Redistribuïm els termes:

$$x_f = x_o + v_o \cdot (t_f - t_o) + \frac{1}{2} \cdot a \cdot (t_f - t_o)^2$$

Suposem que el temps inicial és 0 i ja tenim la funció que buscàvem:

$$x(t) = x_o + v_o \cdot t + \frac{1}{2} \cdot a \cdot t^2$$

Amb aquesta equació podem saber la posició d'un cos en qualsevol instant de temps si està seguint un moviment rectilini uniformement accelerat. Ens és molt útil en programació perquè és la funció que l'ordinador utilitza per calcular la posició d'algun objecte concret cada certs instants de temps.

Existeix una altra manera per arribar a l'equació de la posició del MRUA, fent servir equacions diferencials i càlcul integral. Aquesta explicació figura a l'annex 12.

Abans de passar a veure com es programa el MRUA falta parlar del MCUA, el qual, es basa en el mateix concepte que MRUA: un objecte es mou cada vegada a més velocitat, però en el cas del MCUA ho fa fent voltes al llarg d'una circumferència i a més, igual que passa amb MRU/MCU, canvien els termes en les equacions.

L'equació de la velocitat en funció del temps passa a ser:

$$\omega(t) = \omega_o + \alpha \cdot t$$

**On α és l'acceleració angular en rad/s^2* i ω és la velocitat angular en radiants per segon.*

I l'equació de la posició en funció del temps passa a ser:

$$\varphi(t) = \varphi_o + \omega_o \cdot t + \frac{1}{2} \cdot \alpha \cdot t^2$$

** On ω és la velocitat angular en radiants per segon i φ és l'angle descrit en radiants.*

4.2.2. Programació del principi físic (MRUA/MCUA)

4.2.2.1. MRUA

Per tal de portar el MRUA a la programació hem de fer com en el cas del MRU. Creem un "bucle" amb la instrucció setInterval() i fem que cada segon es repeteixi una funció la qual assigna a un objecte la posició adient segons l'equació del MRUA que acabem de veure: $x(t) = x_o + v_o \cdot t + \frac{1}{2} \cdot a \cdot t^2$

A continuació adjunto un petit simulador de MRUA amb el codi que el fa funcionar:
https://sites.google.com/view/ovsmrua/inicio
Nota: un cop a la pàgina web es pot triar tant la velocitat inicial com l'acceleració del moviment. La posició inicial ja ve determinada.

Codi del Simulador a l'Annex 4.
Fotos del simulador:



Fig. 9. Captures del simulador de MRUA.

4.2.2.1. MCUA

Juntament amb el MRUA hem estat veient el MCUA, el qual es basa en el mateix concepte però descrivint una trajectòria circular. Per tal de programar aquest moviment hem d'utilitzar l'equació: $\varphi(t) = \varphi_o + \omega_o \cdot t + \frac{1}{2} \cdot \alpha \cdot t^2$ i sabent l'angle que s'ha mogut l'objecte podem trobar, utilitzant el radi del moviment que està descrivint i les funcions sinus i cosinus, la posició en la qual ha d'estar. Igual que pel MRUA hem de crear un bucle amb setInterval() que es repeteixi cada segon.

També he programat un simulador per aquest principi físic:
https://sites.google.com/view/mcumcua/inicio

Codi del Simulador a l'Annex 3.

Fotos del simulador:

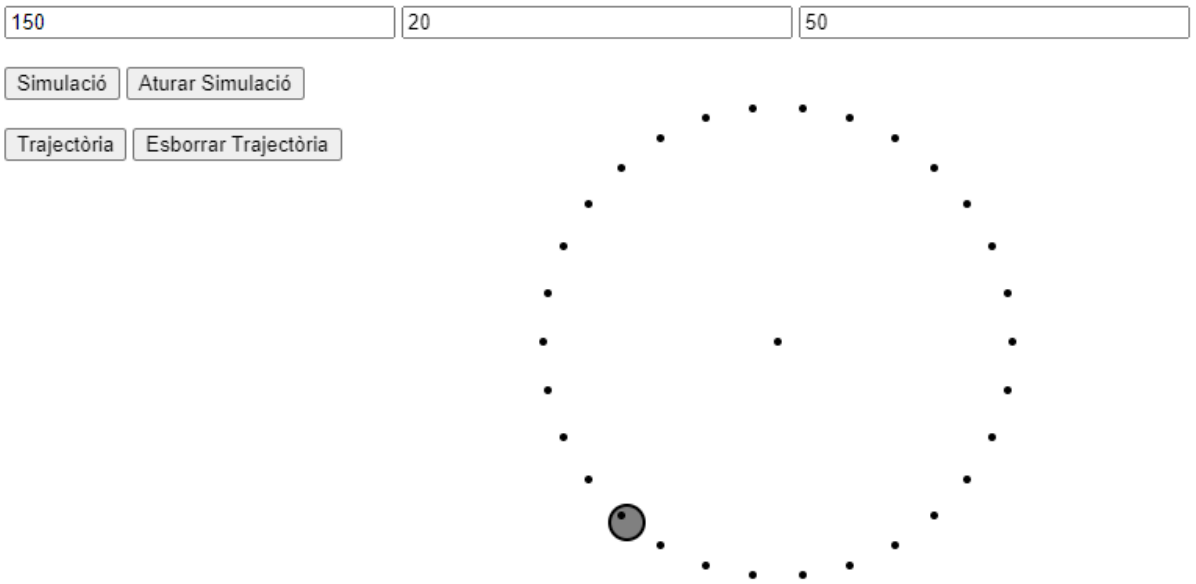


Fig. 10. Captura del simulador de MCU/MCUA

4.2.3. Explicació del principi físic (Tir parabòlic en 2D i 3D)

El tir parabòlic en 2 dimensions és la suma d'un MRU i d'un MRUA i en el cas del tir en 3 dimensions, és la suma de dos MRU i un MRUA.

2D

- El tir parabòlic en dues dimensions ens l'imaginem en un sistema de coordenades amb dos eixos: l'eix x i l'eix y. Per tant, podrem observar un moviment en l'eix x i un moviment en l'eix y, amb velocitats diferents cadascun (v_x i v_y).

Com que suposarem que l'eix x és el terra del planeta en el qual estiguem efectuant el tir, la gravetat (que és una acceleració) anirà cap a terra, per tant, cap a l'eix x i serà negativa segons el sistema de referència de la pàgina següent.

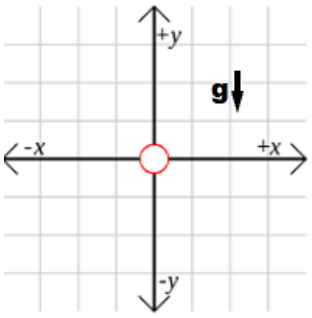


Fig. 11. Sistema de referència x-y per a tirs parabòlics 2D.

El moviment que descriu un objecte en un tir parabòlic en 2 dimensions està definit per les dues equacions següents:

Eix x (MRU):

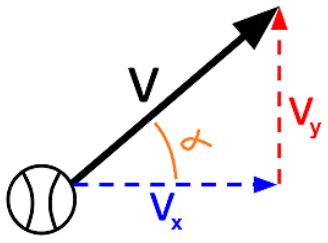
$$x(t) = x_o + v_{ox} \cdot t$$

Eix y (MRUA):

$$y(t) = y_o + v_{oy} \cdot t + \frac{1}{2} \cdot g \cdot t^2$$

**On g és la gravetat:* $-\ 9,8\ m/s.$

Els valors v_x i v_y són causats per la velocitat que porta l'objecte. Si l'objecte porta una velocitat v, llavors v_x és el producte de multiplicar v pel cosinus de l'angle que forma v amb l'eix x (diguem-li α). v_y és el mateix però amb el sinus:



$$v_x = v \cdot cos(\alpha)$$

$$v_y = v \cdot sin(\alpha)$$

Fig. 12. Descomposició d'un vector en els seus components x i y.

Només falta afegir que, com que en l'eix y veiem un MRUA, la v_y varia tal que:

$$v_y(t) = v_{oy} + g \cdot t$$

La v_x és sempre la mateixa

Si tinguéssim els dos components de la velocitat (v_x i v_y) i volguéssim saber quina és la velocitat real de l'objecte (en mòdul) podríem calcular-ho de la següent manera:

$$v = \sqrt{v_x^2 + v_y^2}$$

3D - L'explicació del tir parabòlic en 3D es troba a l'annex 11.

4.2.4. Programació del principi físic (Tir parabòlic en 2D i 3D)

4.2.4.1. Tir parabòlic 2D

En primer lloc, per programar un Tir parabòlic en 2D, hem de recordar el que hem vist anteriorment: el tir parabòlic 2D és la unió d'un MRU en l'eix x i un MRUA en l'eix y. L'únic que cal que fem és un bucle (setInterval) que calculi la posició-x d'un objecte segons la seva velocitat i l'assigni a la propietat "left" (que defineix la posició de l'objecte respecte del marge esquerre de la pantalla). Dins del bucle també haurà de calcular la posició-y segons la velocitat inicial i la gravetat i assignar-la a la qualitat "top" o "bottom" (que defineixen la posició de l'objecte respecte dels marges superior i inferior de la pantalla). A continuació un simulador de tir parabòlic 2D: https://sites.google.com/view/tirparablic/inicio

Codi del Simulador a l'Annex 5.

Foto del simulador:

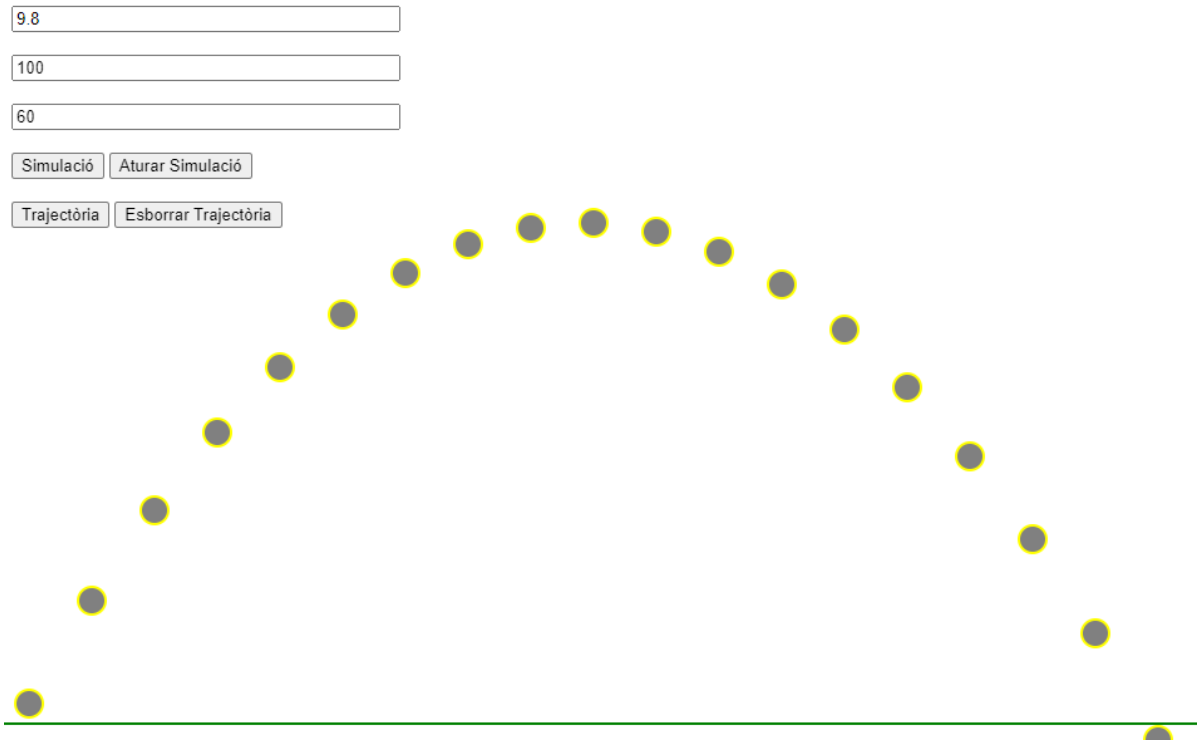


Fig. 13. Captura del simulador de tir parabòlic.

4.2.4.2. Tir parabòlic 3D

En segon lloc, per programar un tir parabòlic 3D, hem de fer el mateix que per un tir 2D el que passa és que hem d'afegir una tercera dimensió, la qual emularem en la pantalla 2D fent que l'objecte es faci més gran o més petit per tal de produir la sensació de profunditat. El temps actua com a tercera dimensió, ja que segons el temps que hagi passat des del principi del moviment, l'objecte serà més petit o més gran en funció de si s'acosta o s'allunya.

4.2.5. Aplicació del principi físic en videojocs

4.2.5.1. Angry Birds 2 (Tir parabòlic 2D)

4.2.5.1.1. Descripció General del videojoc

Angry Birds 2 és un videojoc en el qual s'ha d'intentar colpejar a uns enemics amagats dins de construccions de fusta, vidre i altres materials llençant diferents tipus d'ocells cap a ells. Per llençar els ocells utilitzem un tirador i contra més fort l'estirem a més velocitat surten. A més, movent el ratolí amunt i avall podem controlar la inclinació respecte del terra a la qual sortirà l'ocell. Amb aquestes dues

informacions: la velocitat inicial segons com hàgim estirat el tirador i l'angle inicial segons la posició del ratolí, el videojoc calcula la trajectòria parabòlica que segueix l'ocell sota l'efecte d'una gravetat determinada fins a xocar amb algun altre cos.

Existeixen diferents tipus d'ocells i cadascun amb diferents habilitats especials:
- En primer lloc, tenim l'ocell vermell el qual segueix la trajectòria parabòlica de manera normal i quan cliquem el botó esquerre del ratolí emet una ona expansiva, però això no afecta en la seva trajectòria parabòlica, que és el que ens interessa.
- En segon lloc, l'ocell groc, que també segueix la trajectòria parabòlica de manera normal, però quan fem clic esquerre augmenta la seva velocitat i això fa que la trajectòria sofreix un canvi.
- Per últim, l'ocell blau, que igual que el vermell i el groc: no canvia la seva trajectòria fins que fem clic esquerre, llavors es divideix en tres ocells més petits i cadascun segueix una trajectòria diferent.
** En el joc hi ha molts altres tipus d'ocells, però aquests són els tres principals i els més importants per això només em fixaré en ells.*
He gravat un vídeo on es poden veure les diferents trajectòries que descriuen els ocells.

https://www.youtube.com/watch?v=K-plXXkDBJg

4.2.5.1.2. Estudi de la trajectòria de l'ocell vermell (vegeu trajectòria a l'annex 6):

Per calcular la gravetat i poder predir quina seria la posició de l'ocell vermell en qualsevol moment utilitzaré un programa anomenat "Tracker" en el qual pots introduir vídeos i ell et diu la posició d'un objecte que seleccionis en funció del temps (faré servir un sistema de referència arbitrari però igual per tots els ocells):

A la següent pàgina tenim les dues gràfiques i la taula de valors que he obtingut del "Tracker " per la trajectòria parabòlica de l'ocell vermell.

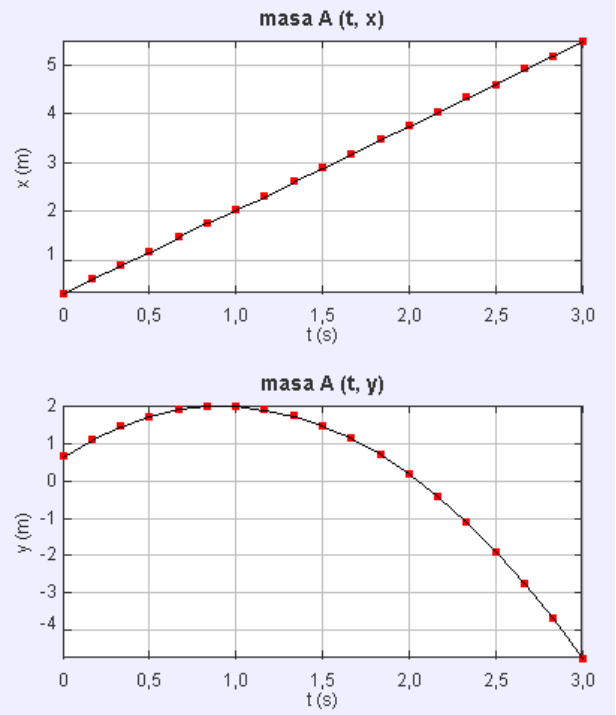


Fig. 14. Gràfiques del moviment en l'eix x (primera gràfica) i en l'eix y (segona gràfica). (extretes de "Tracker").

t (s)	x (m)	y (m)
0,000	0,345	0,673
0,167	0,636	1,127
0,333	0,909	1,473
0,500	1,182	1,745
0,667	1,491	1,945
0,833	1,782	2,018
1,000	2,055	2,036
1,167	2,309	1,927
1,333	2,618	1,764
1,500	2,891	1,491
1,667	3,182	1,164
1,833	3,473	0,727
2,000	3,745	0,200
2,167	4,036	-0,418
2,333	4,327	-1,109
2,500	4,600	-1,891
2,667	4,909	-2,745
2,833	5,182	-3,709
3,000	5,491	-4,764

Fig. 15. Taula de valors relacionada amb les gràfiques. (extreta de "Tracker").

Com podem comprovar, en l'eix x l'ocell segueix un MRU, ja que la posició respecte del temps augmenta uniformement. En l'eix y, en canvi, veiem un MRUA.

Començaré calculant la velocitat en l'eix x, la qual es manté constant. Per fer-ho dividiré una certa distància entre el temps que ha trigat a recórrer-la:

$$v = \frac{\Delta x}{\Delta t} = \frac{3{,}75}{2} = 1{,}875\ m/s$$ *(les dades són extretes de la taula de valors)*

Ara ja sé a quina velocitat es mou l'ocell vermell en l'*eix x*. El següent que necessito saber és la gravetat a la qual està sotmès en el seu moviment en l'*eix y*. Per fer-ho, utilitzaré l'equació de la posició del MRUA:

$$y(t) = y_o + v_{oy} \cdot t + \frac{1}{2} \cdot g \cdot t^2$$

Primer substitueixo a l'equació per dos instants de temps diferents:

Des de "t = 0s" a "t = 2s":

$$0{,}2 = 0{,}67 + v_{oy} \cdot 2 + \frac{1}{2} \cdot g \cdot 2^2$$

Des de "t = 0s" a "t = 3s":

$$-\ 4,76\ =\ 0,67\ +\ v_{oy}\cdot\ 3\ +\ \frac{1}{2}\cdot\ g\ \cdot 3^{2}$$

Utilitzant les dues equacions obtingudes podré saber la velocitat inicial en l'*eix y* i la gravetat:

$$0,2\ -\ 0,67\ =\ 2v_{oy}\ +\ 2g$$

$$-\ 4,76\ -\ 0,67\ =\ 3v_{oy}\ +\ 4,5g$$

Aïllo "g" de la primera:

$$g\ =\ \frac{-\ 0{,}47\ -\ 2v_{oy}}{2}$$

Substitueixo a la segona:

$$-\ 5,43\ =\ 3v_{oy}\ +\ 4,5(\frac{-\ 0{,}47\ -\ 2v_{oy}}{2})\rightarrow v_{oy}\ =\ 2,915\ m/s$$

Trobo "g" que he aïllat anteriorment:

$$g\ =\ \frac{-\ 0{,}47\ -\ 2(2{,}915)}{2}\ =-\ 3,15\ m/s^{2}$$

L'últim que em queda per saber és la velocitat real de l'ocell i l'angle al qual ha sortit respecte de l'*eix x*:

$$v_{o}=\sqrt{v_{ox}^{2}+v_{oy}^{2}}=\sqrt{1,875^{2}+2,915^{2}}=\ 3,466\ m/s$$

$$\alpha_{o}\ =\ arccos(\frac{1{,}875}{3{,}466})\ =\ 57,25º$$

La trajectòria que ha seguit l'ocell vermell està definida per les dues equacions següents:

$$x(t)\ =\ 0,35\ +\ 1,875\ \cdot\ t$$

$$y(t)\ =\ 0,67\ +\ 2,915\ \cdot\ t\ +\ \frac{1}{2}\cdot\ (-\ 3,15)\ \cdot\ t^{2}$$

Tot aquest procediment l'he aplicat també per les trajectòries de l'ocell groc i de l'ocell blau i he obtingut els següents resultats:

Groc: $g = 3,13\ m/s^2$, $v_o = 4,63\ m/s$, $\alpha_o = 60,2º$.

Blau: $g = 3,14\ m/s^2$, $v_o = 5,65\ m/s$, $\alpha_o = 67,3º$.

El procediment detallat l'adjunto a l'annex 8.

4.2.5.2.3. Conclusió (Angry Birds 2)

Les tres gravetats obtingudes són: 3,15 m/s^2 (vermell), 3,13 m/s^2 (groc) i 3,14 m/s^2 (blau), per tant, podríem concloure que la gravetat, simulada pel videojoc, més encertada és 3,14 m/s^2 (la mitjana entre els tres valors). Pel que fa a les velocitats inicials i els angles inicials respecte de l'eix x, no és d'importància que canviïn d'ocell a ocell, ja que les condicions inicials poden ser les que siguin i la gràcia és que el joc simuli correctament la paràbola utilitzant la gravetat.

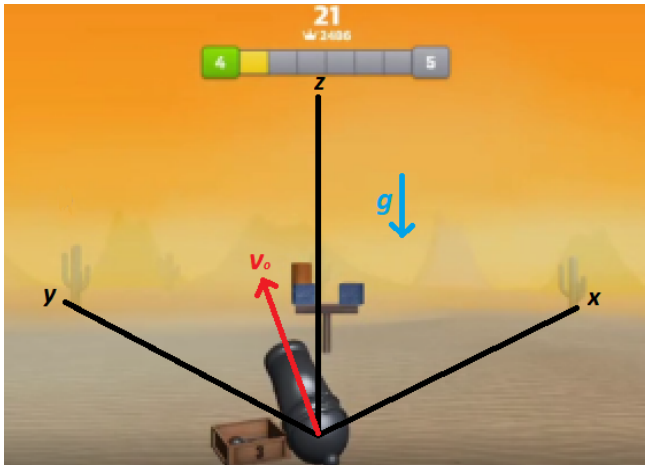
4.2.5.2. Cannon Balls 3D (Tir parabòlic 3D)

Cannon Balls 3D és un joc bastant simple, però aplica principis físics fonamentals: el tir parabòlic 3D i xocs (que els tractaré en l'apartat de "conservació de P").
El joc consisteix en fer caure d'una plataforma construccions de diferents materials fent servir un canó. La idea és igual que en el cas d'Angry Birds 2 però en 3D. Igual que en Angry Birds 2, pots controlar la direcció inicial del projectil, però per altra banda, no pots controlar la velocitat inicial, és sempre la mateixa per tots els llançaments.

He gravat un vídeo en el qual es pot observar l'objectiu del joc i les paràboles que descriuen els projectils.

https://www.youtube.com/watch?v=4GAzGv5vQpI

El videojoc simula una gravetat concreta i per calcular la posició del projectil en cada moment ha de descompondre la velocitat inicial en els tres eixos utilitzant l'angle que tu indiques amb el ratolí. En els eixos x i y calcula la posició com un MRU i en l'eix z com un MRUA:



$$x(t) = 0 + v_{ox} \cdot t$$

$$y(t) = 0 + v_{oy} \cdot t$$

$$z(t) = 0 + v_{oz} \cdot t + \frac{1}{2} \cdot g \cdot t^2$$

Fig. 16. Captura de "Cannon Balls 3D".

A més, com que aquest joc està destinat a jugar-se en pantalles 2D, la tercera dimensió està emulada, vull dir que ens produeix la sensació de veure un entorn 3D, però realment la pantalla és 2D, no té profunditat. Això s'aconsegueix fent que els objectes es facin petits o grans segons estiguin més a prop o més lluny. El videojoc calcula la posició hipotètica que tindria cada cos en un entorn 3D i segons aquesta posició els assigna una mida determinada. Aquesta mida depèn del temps, ja que com més temps hagi passat des del començament del moviment, més lluny es trobarà el projectil i més petit el veurem.

4.2.5.3. Jetpack Joyride (MRUA)

A part d'Angry Birds 2 i Cannon Balls 3D, també he analitzat i observat un joc anomenat Jetpack Joyride, el qual aplica el principi del MRUA. Després d'analitzar-lo he conclòs que el principi estava ben aplicat. Tot el procediment i detalls es troben a l'annex 7.

4.3. Conservació de P (Xocs)

4.3.1. Explicació del principi físic

Quan parlem de xocs en física, estem estudiant la interacció entre dos o més cossos en xocar. Es produeixen xocs en múltiples situacions quotidianes, per exemple, en una partida de billar, quan colpegem amb el pal la bola blanca, estem produint un xoc i quan aquesta bola xoca amb una altra, un altre xoc. També són xocs les explosions o el rebot d'una pilota amb el terra. Per tal d'estudiar totes aquestes interaccions s'utilitza un principi que es basa en la conservació de P.

Quan diem "conservació de P" volem dir conservació de la quantitat de moviment, i què és la quantitat de moviment? És una propietat física que té qualsevol cos i es defineix com:

$$P = m \cdot v \left[\frac{Kg \cdot m}{s}\right]$$

La quantitat de moviment és igual a la massa del cos multiplicada per la velocitat a la qual s'està movent. (Les unitats de P són: $\frac{Kg \cdot m}{s}$)

Aquesta idea que en principi pot semblar poc útil, ens serveix per a resoldre multitud de problemes i sobretot problemes de xocs entre cossos com els que he esmentat anteriorment. Existeixen principalment dos tipus de xocs: els xocs elàstics i els xocs inelàstics:

- En els xocs elàstics ens imaginem un món ideal on no hi ha fregament i tampoc pèrdues d'energia (ja sigui per soroll, per deformacions…) en el xoc.

- Els xocs inelàstics són els que podem veure en el món real, en els quals els cossos s'escalfen, es deformen, fan soroll, en general: perden energia en el xoc.

Tant si parlem de xocs elàstics o inelàstics sabem que un instant abans i un instant després del xoc la quantitat de moviment es manté.

Si un dels cossos té una P de 8 $\frac{Kg \cdot m}{s}$ i l'altre una P de -4 $\frac{Kg \cdot m}{s}$, la P total és de 4 $\frac{Kg \cdot m}{s}$. Podria ser que després del xoc el primer tingues una P de -2 $\frac{Kg \cdot m}{s}$ i el segon una de 6 $\frac{Kg \cdot m}{s}$, amb la qual cosa la P total seguiria sent 4 $\frac{Kg \cdot m}{s}$ i això es compleix en tots els xocs.

Ara bé, en un xoc elàstic es conserva l'energia cinètica però en un xoc inelàstic no, per tant:

Xoc elàstic: $\Delta P = 0$ i $\Delta E_c = 0$

Xoc inelàstic: $\Delta P = 0$ i $\Delta E_c \neq 0$

L'energia cinètica d'un cos es calcula de la següent manera:

$$E_c = \frac{1}{2} \cdot m \cdot v^2$$

Falta per afegir que dins dels xocs inelàstics ens podem trobar xocs 100% inelàstics, en els quals els cossos es queden units després del xoc i van a la mateixa velocitat (també es pot donar, en casos d'explosions, que estiguessin units abans i després no), i xocs parcialment inelàstics, en els quals hi pot haver més o menys pèrdua d'energia però no són ni elàstics ni 100% inelàstics.

Per diferenciar ràpidament els diferents tipus de xocs existeix el "coeficient de restitució" que es calcula a partir de les velocitats dels cossos abans i després del xoc:

$$k = \frac{-(v_1' - v_2')}{v_1 - v_2}$$

** On v_1' i v_2' són les velocitats després del xoc i v_1 i v_2 són les velocitats abans del cos.*

k=0	*100% inelàstic*
1<k<0	*Parcialment inelàstic*
k=1	*Elàstic*

Xocs en 1D

El cas més senzill de xocs que podem trobar és en una dimensió. Els dos cossos es mouen en la mateixa direcció i no cal descompondre la velocitat. Simplement, cal aplicar $\Delta P = 0$ (i $\Delta E_c = 0$ si es dona el cas):

Xocs en 2D

En dues dimensions la cosa es complica una mica, ja que hem de descompondre les velocitats, la quantitat de moviment s'ha de conservar en els dos eixos (x i y) i si es dona un xoc elàstic, l'energia cinètica també s'ha de conservar.

Xocs en 3D

Per últim, en tres dimensions, la velocitat es descompon en tres i la quantitat de moviment s'ha de conservar en l'eix x, en l'eix y i en l'eix z. Si es dona el cas d'un xoc elàstic, l'energia cinètica també s'ha de conservar.

4.3.2. Programació del principi físic

Xocs en 1D

Per tal de programar un xoc en una dimensió, el que caldria fer és situar dos objectes en l'espai i fer que es moguin a certa velocitat (MRU) fins que es trobessin, quan això passes, el programa hauria de calcular les velocitats dels objectes després del xoc i que aquests es comencessin a moure seguint un nou MRU amb les noves velocitats. A continuació es pot observar un simulador d'aquest principi:
https://sites.google.com/view/ovsxoc/inicio
Nota: no posar masses més grans de 500 Kg.

Codi del Simulador a l'Annex 9.

Foto del simulador:



Fig. 17. Captura del simulador de xocs en 1D.

Xocs en 2D

Quan parlem de la programació de xocs en 2D hem de descompondre la velocitat que porten els objectes en els dos eixos i crear un MRU per cada eix. Igual que amb el xoc en 1D, quan els dos cossos es trobin, el programa realitzarà un seguit de càlculs per conèixer la velocitat dels cossos després del xoc i l'angle amb el qual sortiran, per tal que es conservi la quantitat de moviment en els dos eixos. Després de calcular la velocitat i angle de cada cos, els dos emprendran un MRU amb aquelles condicions.

Xocs en 3D

Un xoc en 3D funciona com un xoc en 2D, però hem d'afegir la tercera dimensió, que es calcula amb l'ajuda del temps i canviant les dimensions dels objectes en pantalla per crear una sensació de profunditat. És semblant a la relació entre el tir parabòlic 2D i el tir parabòlic 3D.

4.3.3. Aplicació del principi físic en videojocs

4.3.3.1. Pool Billiards Pro (Conservació de P 2D)

Pool Billiards Pro, un joc de billar per a dispositius mòbils. La idea del joc és fer que la bola blanca es mogui per l'escenari quan la colpeges amb el pal de billar i xoqui amb les altres boles per fer-les entrar en algun dels 6 forats al voltant de la taula de billar. Per fer que les boles no rebotin infinitament, el joc simula fregament amb la

taula i, per tant, quan les boles no estan xocant i s'estan movent de manera recta, estan descrivint un MRUA amb acceleració negativa. El joc va actualitzant la velocitat que duen les boles cada cert temps i quan es produeix un xoc entre dues voles, utilitza les velocitats que duien just abans del xoc per calcular, mitjançant la conservació de la quantitat de moviment que hem vist anteriorment, les velocitats que duran després del xoc.

Per tal d'entendre més clarament el que acabo d'explicar, he gravat un vídeo on es poden veure diversos xocs.

https://www.youtube.com/watch?v=HthXadHMcpM&t=5s

Cal afegir que tots els xocs que s'efectuen en aquest joc, són en dues dimensions, per tant, la quantitat de moviment es conserva en els dos eixos.

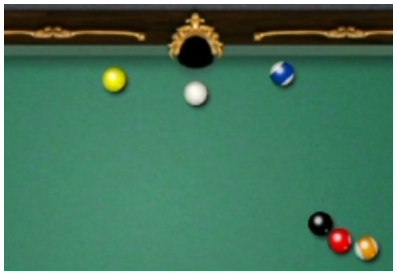


Fig. 18. Captura de "Pool Billiards Pro".

En aquesta imatge, la bola blanca està a punt de xocar amb la bola groga i quan ho faci, el joc tindrà en compte l'angle respecte l'eix *x* al qual s'estava movent la bola per descompondre la velocitat i poder calcular la conservació de P en els dos eixos.

Després, trobarà les components de les noves velocitats i calculant-ne el mòdul ja sabrà a quina velocitat es mouen la bola groga i la bola blanca just després del xoc, moment en el qual començarà un MRUA de frenada fins que les boles s'aturin.

4.3.3.2. Angry Birds 2 (Conservació de P 2D) i Cannon Balls 3D (Conservació de P 3D)

Aquests dos videojocs (Angry Birds 2 i Cannon Balls 3D) ja han sortit anteriorment en el treball quan parlava sobre MRUA i tir parabòlic, però resulta que a part d'aplicar aquell principi físic també apliquen la conservació de P per a simular xocs i ho fan correctament. Hi ha una explicació més extensa a l'annex 10.

5. Conclusions

Les meves conclusions després de realitzar aquest treball són les següents:

1. En primer lloc, he pogut comprovar que els videojocs contenen principis físics, sobretot de cinemàtica (en aquests he centrat el meu treball), però, tot i això, també hi ha videojocs que apliquen altres principis físics, per exemple, d'il·luminació, per la generació de reflexos i ombres.

2. En segon lloc, he comprovat que aquells videojocs on crec que la cinemàtica es veu amb més claredat: Geometry Dash, Tap on Time, Pac-Man, Jetpack Joyride, Angry Birds 2, Cannon Balls 3D i Pool Billiards Pro, apliquen els principis físics, que intenten simular (MRU, MCU, MRUA, Tir parabòlic, i Conservació de P), de manera correcta.

3. En tercer lloc, com a part pràctica, he après els tres llenguatges de programació HTML, CSS i JavaScript, amb la profunditat necessària pel que havia de programar i he pogut realitzar diversos simuladors que apliquen els principis físics treballats.

En general, he complert els objectius inicials del meu treball, he dut a terme una part teòrica amb resultats coherents i correctes en la qual he vist que Geometry Dash i Pac-Man apliquen correctament el principi físic MRU, Tap on Time aplica MCU, Angry Birds 2, Jetpack Joyride i Cannon Balls 3D apliquen correctament el tir parabòlic i Pool Billiards Pro juntament amb, una altra vegada, Angry Birds 2 i Cannon Balls 3D apliquen la conservació de la quantitat de moviment; i, per altra banda, també he programat cinc simuladors funcionals de MRU, MCU/MCUA, tir parabòlic i xocs en 1D.

6. Webgrafia

Aprenentatge de llenguatges de programació:

https://www.w3schools.com/

https://www.youtube.com/watch?v=rbuYtrNUxg4

https://www.youtube.com/watch?v=Q9fwkpxr3Dw

Història dels videojocs:

https://www.fib.upc.edu/retroinformatica/historia/videojocs.html

https://www.petitsapiens.cat/un-viatge-al-passat/historia-videojocs_203149_102.html

Derivació/Integració per la resolució de funcions diferencials:

https://www.disfrutalasmatematicas.com/calculo/integracion-reglas.html

https://elpinguinotolkiano.wordpress.com/2014/09/29/calculo-diferencial-2-posiciones-velocidades-y-aceleraciones/

Explicació dels principis físics:

https://www.fisicalab.com/apartado/mru

https://www.fisicalab.com/apartado/mrua-ecuaciones

https://www.fisicalab.com/apartado/conservacion-momento-lineal

https://www.fisicalab.com/apartado/ecuaciones-mcua